МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«Санкт-Петербургский государственный университет аэрокосмического приборостроения»

КАФЕДРА № 12

ОТЧЕТ

о лабораторной работе № 1

Разработка системы классов «Книжный магазин» и контейнера Keeper

Вариант 8

Дисциплина: _____

Выполнил: студент группы 1423
Камина Е. В.

Проверил: _____

Дата защиты: _____

Санкт-Петербург
2026

1 Постановка задачи

Цель работы — разработать программу на языке C++ с применением наследования, полиморфизма, динамического выделения памяти и работы с файлами.

Вариант 8. Класс «Книжный магазин» хранит сведения о книгах, учебниках и канцелярских товарах. Для книги указываются название, автор, год выпуска, аннотация, жанр, количество страниц и стоимость. Для учебника указываются название, автор, год выпуска, учебное заведение, год обучения, количество страниц и стоимость. Для канцелярии указываются вид, цвет, назначение и стоимость.

Необходимо создать абстрактный класс Base и производные классы по трём типам товаров. Контейнер Keeper должен добавлять и удалять объекты, полностью сохранять содержимое в файл и полностью восстанавливать его из файла.

Дополнительные требования: пользовательское меню, вывод и редактирование каждого типа товаров, динамическая память без STL-контейнеров, конструкторы без параметров, с параметрами и копирования, деструкторы с сообщениями, перегрузка операторов, get/set-методы, исключения с текстом ошибок и раздельные файлы .h/.cpp для каждого класса. Функция main находится в отдельном файле.

2 Формализация задачи

Каждый товар магазина представляет собой объект одного из трёх производных классов. Общее свойство всех товаров — стоимость — хранится в классе Base. Основные данные конкретных товаров находятся в производных классах.

Контейнер Keeper хранит указатели на объекты Base: items[0], items[1], ..., items[n − 1]. При этом n — текущее количество товаров, capacity — размер выделенного массива. Всегда выполняется $0 \le n \le capacity$. Если при добавлении массив заполнен, выделяется новый массив с удвоенной вместимостью; указатели переносятся в него, старый массив указателей удаляется.

При удалении выбранный объект удаляется оператором delete, остальные указатели сдвигаются на одну позицию влево. Для копирования каждого объекта применяется виртуальная функция clone(), благодаря которой сохраняется конкретный тип товара.

При сохранении в текстовый файл сначала записываются заголовок формата и число товаров, затем тип каждого товара и его поля. При загрузке по номеру типа создаётся объект нужного класса, после чего читаются его значения. Данные сначала загружаются во временный Keeper: если файл содержит ошибку, старое содержимое магазина не теряется.

3 Таблица описания классов

Разработаны пять основных классов. Их назначение и связи приведены в таблице 1.

Таблица 1 — Описание классов

Класс	Назначение	Связь	Основные данные и методы
Base	Абстрактный класс товара	Родительский класс	Стоимость; getPrice, setPrice; виртуальные print, input, edit, save, load, clone.
Book	Обычная книга	public-наследование Base	Название, автор, год, аннотация, жанр, страницы; get/set.
Textbook	Учебник	public-наследование Base	Название, автор, год, учебное заведение, год обучения, страницы; get/set.
Stationery	Канцелярский товар	public-наследование Base	Вид, цвет, назначение; get/set.
Keeper	Хранение всех товаров	Композиция: владеет Base*	Динамический массив Base**, add, remove, showAll, сохранение/чтение файлов.

Все три производных класса переопределяют виртуальные функции Base. Keeper хранит адреса объектов разных типов и управляет сроком их жизни.

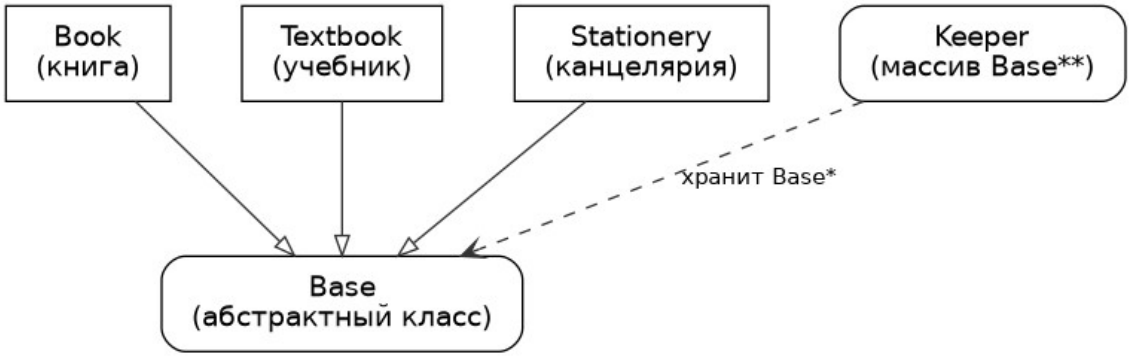


Рисунок 1 — Иерархия классов и связь Keeper с объектами

На рисунке 1 сплошные стрелки показывают наследование классов Book, Textbook и Stationery от Base. Пунктирная связь показывает, что Keeper хранит указатели на объекты базового класса и удаляет их при необходимости.

4 Исходный код

Программа написана на C++17 и разделена на файлы объявлений .h и реализации .cpp. Функция main находится в файле main.cpp. Пользовательский ввод вынесен в Input.h и Input.cpp, чтобы не повторять одинаковые проверки.

Основные операции с памятью реализованы в Keeper.cpp. Функция add() расширяет массив при необходимости, remove() освобождает выбранный объект, конструктор копирования создаёт независимые копии товаров с помощью clone(). Все основные файлы исходного кода представлены полностью в приложении А.

Примеры перегрузки операторов: Keeper::operator+= добавляет товар, Keeper::operator[] предоставляет доступ к товару по индексу, Keeper::operator= выполняет глубокое копирование, а operator<< выводит товар с помощью виртуальной функции print().

5 Результаты работы программы

Программа была собрана компилятором g++ в режиме C++17. Выполнены сценарии добавления всех трёх типов товаров, просмотра, редактирования, удаления, сохранения и загрузки, а также копирования контейнера. Примеры поведения приведены ниже.

Таблица 2 — Проверка основных операций

Действие	Результат	Комментарий
Добавление трёх товаров	Все товары отображаются	Хранятся объекты разных производных классов.
Изменение цвета ручки	Цвет меняется с Blue на Black	Используется индивидуальный метод edit().
Сохранение в файл	All products saved.	Сохраняются все объекты с их типами и полями.
Удаление и загрузка	All products loaded.	Товар после восстановления появляется снова.
Копирование Keeper	Copied objects: 3; Assigned objects: 3	Создаются независимые копии всех объектов.
Повреждённый файл	Выводится Error, старые данные сохраняются	Проверяется механизм обработки исключений.

Таблица 2 показывает, что пользователь может работать со всеми видами товаров. Особенно важно, что данные не теряются после сохранения и загрузки, а ошибка в файле обрабатывается без удаления уже введённых товаров.

Пример фрагмента вывода при просмотре товаров:

```
--- Product #1 ---
Type: Book
Title: The Master and Margarita
Author: Mikhail Bulgakov
Year: 1967
Pages: 480
Price: 790.5

--- Product #3 ---
Type: Stationery
Stationery: Pen
Color: Black
Purpose: Writing
Price: 45
```

После выбора команды 6 пользователь изменил цвет ручки на Black. Затем данные были сохранены командой 8. После удаления одной книги и повторной загрузки командой 9 восстановились все три объекта.

Конструкторы и деструкторы выводят диагностические сообщения вида [Constructor] Book: default и [Destructor] Book: ..., поэтому можно проследить создание и уничтожение объектов. Исключения возникают при некорректных данных (например, отрицательной цене) или отсутствии файла. Они сопровождаются поясняющим сообщением.

6 Выводы

В лабораторной работе разработана консольная программа «Книжный магазин» для хранения и изменения сведений о товарах. Реализованы наследование от абстрактного базового класса, виртуальные функции и полиморфизм.

Создан контейнер Keeper на основе динамически выделяемого массива указателей без применения STL-контейнеров. Реализованы сохранение и загрузка объектов из файла, конструкторы и деструкторы, глубокое копирование, перегрузка операторов, методы get/set и обработка исключительных ситуаций. Полученные результаты соответствуют предусмотренным операциям варианта 8.

7 Ссылка на репозиторий

Репозиторий GitHub (вставить после публикации проекта):

_____

По требованиям задания исходный код должен быть размещён на GitHub или Bitbucket. Необходимо сделать не менее трёх коммитов, разнесённых во времени, с локального компьютера, и после этого указать здесь действующую ссылку на проект.

Список использованных источников:

1. Лабораторная работа №1: методические указания и варианты индивидуальных заданий (предоставленный преподавателем PDF-файл).

2. Нормативная документация ГУАП. Для учебного процесса. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 17.10.2026).

ПРИЛОЖЕНИЕ А. Полный исходный код программы

Ниже приведены все файлы исходного кода проекта. Имена и порядок файлов соответствуют структуре каталога src.

Файл Base.h

```
#ifndef BASE_H
#define BASE_H

#include <iosfwd>

// Abstract parent for all products of the shop.
class Base {
private:
    double price_;

public:
    Base();
    explicit Base(double price);
    Base(const Base& other);
    virtual ~Base();

    double getPrice() const;
    void setPrice(double price);

    virtual int typeId() const = 0;
    virtual const char* typeName() const = 0;
    virtual void input() = 0;
    virtual void edit() = 0;
    virtual void print(std::ostream& out) const = 0;
    virtual void save(std::ostream& out) const = 0;
    virtual void load(std::istream& in) = 0;
    virtual Base* clone() const = 0;
};

std::ostream& operator<<(std::ostream& out, const Base& item);

#endif
```

Файл Base.cpp

```
#include "Base.h"

#include <cmath>
#include <iostream>
#include <stdexcept>

Base::Base() : price_(0.0) {
    std::cout << "[Constructor] Base: default\n";
}

Base::Base(double price) : price_(0.0) {
    setPrice(price);
    std::cout << "[Constructor] Base: with parameters\n";
}

Base::Base(const Base& other) : price_(other.price_) {
    std::cout << "[Constructor] Base: copy\n";
}

Base::~Base() {
    std::cout << "[Destructor] Base\n";
}

double Base::getPrice() const {
    return price_;
}

void Base::setPrice(double price) {
    if (!std::isfinite(price) || price < 0.0) {
        throw std::invalid_argument("Price cannot be negative or invalid.");
    }
    price_ = price;
```

```
}

std::ostream& operator<<(std::ostream& out, const Base& item) {
    item.print(out);
    return out;
}
```

Файл Book.h

```
#ifndef BOOK_H
#define BOOK_H

#include "Base.h"
#include <string>

class Book : public Base {
private:
    std::string title_;
    std::string author_;
    int year_;
    std::string annotation_;
    std::string genre_;
    int pages_;

public:
    Book();
    Book(const std::string& title, const std::string& author, int year,
         const std::string& annotation, const std::string& genre,
         int pages, double price);
    Book(const Book& other);
    ~Book() override;

    const std::string& getTitle() const;
    const std::string& getAuthor() const;
    int getYear() const;
    const std::string& getAnnotation() const;
    const std::string& getGenre() const;
    int getPages() const;

    void setTitle(const std::string& title);
    void setAuthor(const std::string& author);
    void setYear(int year);
    void setAnnotation(const std::string& annotation);
    void setGenre(const std::string& genre);
    void setPages(int pages);

    int typeId() const override;
    const char* typeName() const override;
    void input() override;
    void edit() override;
    void print(std::ostream& out) const override;
    void save(std::ostream& out) const override;
    void load(std::istream& in) override;
    Base* clone() const override;
};

#endif
```

Файл Book.cpp

```
#include "Book.h"
#include "Input.h"

#include <climits>
#include <iomanip>
#include <iostream>
#include <stdexcept>

Book::Book() : Base(), year_(1), pages_(1) {
    std::cout << "[Constructor] Book: default\n";
}

Book::Book(const std::string& title, const std::string& author, int year,
           const std::string& annotation, const std::string& genre,
           int pages, double price)
    : Base(price), year_(1), pages_(1) {
```

```
    setTitle(title);
    setAuthor(author);
    setYear(year);
    setAnnotation(annotation);
    setGenre(genre);
    setPages(pages);
    std::cout << "[Constructor] Book: with parameters\n";
}

Book::Book(const Book& other)
    : Base(other), title_(other.title_), author_(other.author_),
      year_(other.year_), annotation_(other.annotation_),
      genre_(other.genre_), pages_(other.pages_) {
    std::cout << "[Constructor] Book: copy\n";
}

Book::~Book() {
    std::cout << "[Destructor] Book: " << title_ << "\n";
}

const std::string& Book::getTitle() const { return title_; }
const std::string& Book::getAuthor() const { return author_; }
int Book::getYear() const { return year_; }
const std::string& Book::getAnnotation() const { return annotation_; }
const std::string& Book::getGenre() const { return genre_; }
int Book::getPages() const { return pages_; }

void Book::setTitle(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Book title is empty.");
    title_ = value;
}
void Book::setAuthor(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Book author is empty.");
    author_ = value;
}
void Book::setYear(int value) {
    if (value < 1) throw std::invalid_argument("Year must be positive.");
    year_ = value;
}
void Book::setAnnotation(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Annotation is empty.");
    annotation_ = value;
}
void Book::setGenre(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Genre is empty.");
    genre_ = value;
}
void Book::setPages(int value) {
    if (value < 1) throw std::invalid_argument("Pages must be positive.");
    pages_ = value;
}

int Book::typeId() const { return 1; }
const char* Book::typeName() const { return "Book"; }

void Book::input() {
    setTitle(readText("Title: "));
    setAuthor(readText("Author: "));
    setYear(readInt("Publication year: ", 1, INT_MAX));
    setAnnotation(readText("Annotation: "));
    setGenre(readText("Genre: "));
    setPages(readInt("Page count: ", 1, INT_MAX));
    setPrice(readPrice("Price: "));
}

void Book::edit() {
    std::cout << "1 Title | 2 Author | 3 Year | 4 Annotation | "
                 "5 Genre | 6 Pages | 7 Price\n";
    int option = readInt("Field to change: ", 1, 7);
    switch (option) {
        case 1: setTitle(readText("New title: ")); break;
        case 2: setAuthor(readText("New author: ")); break;
        case 3: setYear(readInt("New year: ", 1, INT_MAX)); break;
        case 4: setAnnotation(readText("New annotation: ")); break;
        case 5: setGenre(readText("New genre: ")); break;
```

```
        case 6: setPages(readInt("New page count: ", 1, INT_MAX)); break;
        case 7: setPrice(readPrice("New price: ")); break;
    }
}

void Book::print(std::ostream& out) const {
    out << "Type: " << typeName() << "\n"
        << "Title: " << title_ << "\n"
        << "Author: " << author_ << "\n"
        << "Year: " << year_ << "\n"
        << "Annotation: " << annotation_ << "\n"
        << "Genre: " << genre_ << "\n"
        << "Pages: " << pages_ << "\n"
        << "Price: " << getPrice() << "\n";
}

void Book::save(std::ostream& out) const {
    out << std::quoted(title_) << '\n'
        << std::quoted(author_) << '\n'
        << year_ << '\n'
        << std::quoted(annotation_) << '\n'
        << std::quoted(genre_) << '\n'
        << pages_ << '\n'
        << getPrice() << '\n';
}

void Book::load(std::istream& in) {
    std::string title, author, annotation, genre;
    int year, pages;
    double price;
    if (!(in >> std::quoted(title) >> std::quoted(author) >> year
          >> std::quoted(annotation) >> std::quoted(genre) >> pages >> price)) {
        throw std::runtime_error("Cannot read book data from file.");
    }
    setTitle(title);
    setAuthor(author);
    setYear(year);
    setAnnotation(annotation);
    setGenre(genre);
    setPages(pages);
    setPrice(price);
}

Base* Book::clone() const {
    return new Book(*this);
}
```

Файл Textbook.h

```
#ifndef TEXTBOOK_H
#define TEXTBOOK_H

#include "Base.h"
#include <string>

class Textbook : public Base {
private:
    std::string title_;
    std::string author_;
    int year_;
    std::string institution_;
    int studyYear_;
    int pages_;

public:
    Textbook();
    Textbook(const std::string& title, const std::string& author, int year,
             const std::string& institution, int studyYear,
             int pages, double price);
    Textbook(const Textbook& other);
    ~Textbook() override;

    const std::string& getTitle() const;
    const std::string& getAuthor() const;
    int getYear() const;
```

```
    const std::string& getInstitution() const;
    int getStudyYear() const;
    int getPages() const;

    void setTitle(const std::string& value);
    void setAuthor(const std::string& value);
    void setYear(int value);
    void setInstitution(const std::string& value);
    void setStudyYear(int value);
    void setPages(int value);

    int typeId() const override;
    const char* typeName() const override;
    void input() override;
    void edit() override;
    void print(std::ostream& out) const override;
    void save(std::ostream& out) const override;
    void load(std::istream& in) override;
    Base* clone() const override;
};

#endif
```

Файл Textbook.cpp

```
#include "Textbook.h"
#include "Input.h"

#include <climits>
#include <iomanip>
#include <iostream>
#include <stdexcept>

Textbook::Textbook() : Base(), year_(1), studyYear_(1), pages_(1) {
    std::cout << "[Constructor] Textbook: default\n";
}

Textbook::Textbook(const std::string& title, const std::string& author,
                   int year, const std::string& institution, int studyYear,
                   int pages, double price)
    : Base(price), year_(1), studyYear_(1), pages_(1) {
    setTitle(title);
    setAuthor(author);
    setYear(year);
    setInstitution(institution);
    setStudyYear(studyYear);
    setPages(pages);
    std::cout << "[Constructor] Textbook: with parameters\n";
}

Textbook::Textbook(const Textbook& other)
    : Base(other), title_(other.title_), author_(other.author_),
      year_(other.year_), institution_(other.institution_),
      studyYear_(other.studyYear_), pages_(other.pages_) {
    std::cout << "[Constructor] Textbook: copy\n";
}

Textbook::~Textbook() {
    std::cout << "[Destructor] Textbook: " << title_ << "\n";
}

const std::string& Textbook::getTitle() const { return title_; }
const std::string& Textbook::getAuthor() const { return author_; }
int Textbook::getYear() const { return year_; }
const std::string& Textbook::getInstitution() const { return institution_; }
int Textbook::getStudyYear() const { return studyYear_; }
int Textbook::getPages() const { return pages_; }

void Textbook::setTitle(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Textbook title is empty.");
    title_ = value;
}
void Textbook::setAuthor(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Textbook author is empty.");
    author_ = value;
```

```
}
void Textbook::setYear(int value) {
    if (value < 1) throw std::invalid_argument("Year must be positive.");
    year_ = value;
}
void Textbook::setInstitution(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Institution is empty.");
    institution_ = value;
}
void Textbook::setStudyYear(int value) {
    if (value < 1) throw std::invalid_argument("Study year must be positive.");
    studyYear_ = value;
}
void Textbook::setPages(int value) {
    if (value < 1) throw std::invalid_argument("Pages must be positive.");
    pages_ = value;
}

int Textbook::typeId() const { return 2; }
const char* Textbook::typeName() const { return "Textbook"; }

void Textbook::input() {
    setTitle(readText("Title: "));
    setAuthor(readText("Author: "));
    setYear(readInt("Publication year: ", 1, INT_MAX));
    setInstitution(readText("Educational institution: "));
    setStudyYear(readInt("Study year / course: ", 1, INT_MAX));
    setPages(readInt("Page count: ", 1, INT_MAX));
    setPrice(readPrice("Price: "));
}

void Textbook::edit() {
    std::cout << "1 Title | 2 Author | 3 Year | 4 Institution | "
                 "5 Study year | 6 Pages | 7 Price\n";
    int option = readInt("Field to change: ", 1, 7);
    switch (option) {
        case 1: setTitle(readText("New title: ")); break;
        case 2: setAuthor(readText("New author: ")); break;
        case 3: setYear(readInt("New publication year: ", 1, INT_MAX)); break;
        case 4: setInstitution(readText("New institution: ")); break;
        case 5: setStudyYear(readInt("New study year: ", 1, INT_MAX)); break;
        case 6: setPages(readInt("New page count: ", 1, INT_MAX)); break;
        case 7: setPrice(readPrice("New price: ")); break;
    }
}

void Textbook::print(std::ostream& out) const {
    out << "Type: " << typeName() << "\n"
        << "Title: " << title_ << "\n"
        << "Author: " << author_ << "\n"
        << "Year: " << year_ << "\n"
        << "Institution: " << institution_ << "\n"
        << "Study year: " << studyYear_ << "\n"
        << "Pages: " << pages_ << "\n"
        << "Price: " << getPrice() << "\n";
}

void Textbook::save(std::ostream& out) const {
    out << std::quoted(title_) << '\n'
        << std::quoted(author_) << '\n'
        << year_ << '\n'
        << std::quoted(institution_) << '\n'
        << studyYear_ << '\n'
        << pages_ << '\n'
        << getPrice() << '\n';
}

void Textbook::load(std::istream& in) {
    std::string title, author, institution;
    int year, studyYear, pages;
    double price;
    if (!(in >> std::quoted(title) >> std::quoted(author) >> year
          >> std::quoted(institution) >> studyYear >> pages >> price)) {
        throw std::runtime_error("Cannot read textbook data from file.");
    }
```

```
    setTitle(title);
    setAuthor(author);
    setYear(year);
    setInstitution(institution);
    setStudyYear(studyYear);
    setPages(pages);
    setPrice(price);
}

Base* Textbook::clone() const {
    return new Textbook(*this);
}
```

Файл Stationery.h

```
#ifndef STATIONERY_H
#define STATIONERY_H

#include "Base.h"
#include <string>

class Stationery : public Base {
private:
    std::string kind_;
    std::string color_;
    std::string purpose_;

public:
    Stationery();
    Stationery(const std::string& kind, const std::string& color,
               const std::string& purpose, double price);
    Stationery(const Stationery& other);
    ~Stationery() override;

    const std::string& getKind() const;
    const std::string& getColor() const;
    const std::string& getPurpose() const;
    void setKind(const std::string& value);
    void setColor(const std::string& value);
    void setPurpose(const std::string& value);

    int typeId() const override;
    const char* typeName() const override;
    void input() override;
    void edit() override;
    void print(std::ostream& out) const override;
    void save(std::ostream& out) const override;
    void load(std::istream& in) override;
    Base* clone() const override;
};

#endif
```

Файл Stationery.cpp

```
#include "Stationery.h"
#include "Input.h"

#include <iomanip>
#include <iostream>
#include <stdexcept>

Stationery::Stationery() : Base() {
    std::cout << "[Constructor] Stationery: default\n";
}

Stationery::Stationery(const std::string& kind, const std::string& color,
                       const std::string& purpose, double price)
    : Base(price) {
    setKind(kind);
    setColor(color);
    setPurpose(purpose);
    std::cout << "[Constructor] Stationery: with parameters\n";
}

Stationery::Stationery(const Stationery& other)
```

```
    : Base(other), kind_(other.kind_), color_(other.color_),
      purpose_(other.purpose_) {
    std::cout << "[Constructor] Stationery: copy\n";
}

Stationery::~Stationery() {
    std::cout << "[Destructor] Stationery: " << kind_ << "\n";
}

const std::string& Stationery::getKind() const { return kind_; }
const std::string& Stationery::getColor() const { return color_; }
const std::string& Stationery::getPurpose() const { return purpose_; }

void Stationery::setKind(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Stationery type is empty.");
    kind_ = value;
}
void Stationery::setColor(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Color is empty.");
    color_ = value;
}
void Stationery::setPurpose(const std::string& value) {
    if (value.empty()) throw std::invalid_argument("Purpose is empty.");
    purpose_ = value;
}

int Stationery::typeId() const { return 3; }
const char* Stationery::typeName() const { return "Stationery"; }

void Stationery::input() {
    setKind(readText("Stationery type (pen, notebook, etc.): "));
    setColor(readText("Color: "));
    setPurpose(readText("Purpose: "));
    setPrice(readPrice("Price: "));
}

void Stationery::edit() {
    std::cout << "1 Type | 2 Color | 3 Purpose | 4 Price\n";
    int option = readInt("Field to change: ", 1, 4);
    switch (option) {
        case 1: setKind(readText("New type: ")); break;
        case 2: setColor(readText("New color: ")); break;
        case 3: setPurpose(readText("New purpose: ")); break;
        case 4: setPrice(readPrice("New price: ")); break;
    }
}

void Stationery::print(std::ostream& out) const {
    out << "Type: " << typeName() << "\n"
        << "Stationery: " << kind_ << "\n"
        << "Color: " << color_ << "\n"
        << "Purpose: " << purpose_ << "\n"
        << "Price: " << getPrice() << "\n";
}

void Stationery::save(std::ostream& out) const {
    out << std::quoted(kind_) << '\n'
        << std::quoted(color_) << '\n'
        << std::quoted(purpose_) << '\n'
        << getPrice() << '\n';
}

void Stationery::load(std::istream& in) {
    std::string kind, color, purpose;
    double price;
    if (!(in >> std::quoted(kind) >> std::quoted(color)
          >> std::quoted(purpose) >> price)) {
        throw std::runtime_error("Cannot read stationery data from file.");
    }
    setKind(kind);
    setColor(color);
    setPurpose(purpose);
    setPrice(price);
}
```

```
Base* Stationery::clone() const {
    return new Stationery(*this);
}
```

Файл Keeper.h

```
#ifndef KEEPER_H
#define KEEPER_H

#include "Base.h"
#include <string>

// Owns a manually allocated, growing array of Base* (no STL containers).
class Keeper {
private:
    Base** items_;
    int count_;
    int capacity_;

    void reserve(int newCapacity);
    void swapWith(Keeper& other);

public:
    Keeper();
    explicit Keeper(int capacity);
    Keeper(const Keeper& other);
    ~Keeper();

    Keeper& operator=(const Keeper& other);
    Keeper& operator+=(Base* item);
    Base& operator[](int index);
    const Base& operator[](int index) const;

    int size() const;
    void add(Base* item);
    void remove(int index);
    void clear();
    void showAll() const;
    void saveToFile(const std::string& filename) const;
    void loadFromFile(const std::string& filename);
};

#endif
```

Файл Keeper.cpp

```
#include "Keeper.h"
#include "Book.h"
#include "Stationery.h"
#include "Textbook.h"

#include <fstream>
#include <iostream>
#include <limits>
#include <stdexcept>

Keeper::Keeper() : items_(new Base*[2]), count_(0), capacity_(2) {
    std::cout << "[Constructor] Keeper: default\n";
}

Keeper::Keeper(int capacity) : items_(nullptr), count_(0), capacity_(capacity) {
    if (capacity < 1) throw std::invalid_argument("Capacity must be positive.");
    items_ = new Base*[capacity_];
    std::cout << "[Constructor] Keeper: with parameters\n";
}

Keeper::Keeper(const Keeper& other)
    : items_(new Base*[other.capacity_]), count_(0), capacity_(other.capacity_) {
    try {
        for (int i = 0; i < other.count_; ++i) {
            items_[count_++] = other.items_[i]->clone();
        }
    } catch (...) {
        clear();
        delete[] items_;
        throw;
```

```
    }
    std::cout << "[Constructor] Keeper: copy\n";
}

Keeper::~Keeper() {
    clear();
    delete[] items_;
    std::cout << "[Destructor] Keeper\n";
}

void Keeper::clear() {
    for (int i = 0; i < count_; ++i) {
        delete items_[i];
    }
    count_ = 0;
}

void Keeper::swapWith(Keeper& other) {
    Base** oldItems = items_;
    items_ = other.items_;
    other.items_ = oldItems;

    int oldCount = count_;
    count_ = other.count_;
    other.count_ = oldCount;

    int oldCapacity = capacity_;
    capacity_ = other.capacity_;
    other.capacity_ = oldCapacity;
}

Keeper& Keeper::operator=(const Keeper& other) {
    if (this != &other) {
        Keeper copy(other);
        swapWith(copy);
    }
    return *this;
}

Keeper& Keeper::operator+=(Base* item) {
    add(item);
    return *this;
}

int Keeper::size() const { return count_; }

void Keeper::reserve(int newCapacity) {
    if (newCapacity <= capacity_) return;
    Base** newItems = new Base*[newCapacity];
    for (int i = 0; i < count_; ++i) {
        newItems[i] = items_[i];
    }
    delete[] items_;
    items_ = newItems;
    capacity_ = newCapacity;
}

void Keeper::add(Base* item) {
    if (item == nullptr) throw std::invalid_argument("Cannot add null object.");
    if (count_ == capacity_) {
        if (capacity_ > std::numeric_limits<int>::max() / 2) {
            throw std::runtime_error("Too many objects in Keeper.");
        }
        reserve(capacity_ * 2);
    }
    items_[count_++] = item; // Keeper owns the object from this point.
}

Base& Keeper::operator[](int index) {
    if (index < 0 || index >= count_) {
        throw std::out_of_range("There is no product with this number.");
    }
    return *items_[index];
}
```

```
const Base& Keeper::operator[](int index) const {
    if (index < 0 || index >= count_) {
        throw std::out_of_range("There is no product with this number.");
    }
    return *items_[index];
}

void Keeper::remove(int index) {
    if (index < 0 || index >= count_) {
        throw std::out_of_range("Cannot delete: invalid product number.");
    }
    delete items_[index];
    for (int i = index; i < count_ - 1; ++i) {
        items_[i] = items_[i + 1];
    }
    --count_;
}

void Keeper::showAll() const {
    if (count_ == 0) {
        std::cout << "The shop is empty.\n";
        return;
    }
    for (int i = 0; i < count_; ++i) {
        std::cout << "\n--- Product #" << (i + 1) << " ---\n"
                  << *items_[i];
    }
}

void Keeper::saveToFile(const std::string& filename) const {
    std::ofstream out(filename.c_str());
    if (!out) throw std::runtime_error("Cannot open file for writing.");
    out.precision(17);
    out << "BOOKSTORE_LAB1_V1\n" << count_ << '\n';
    for (int i = 0; i < count_; ++i) {
        out << items_[i]->typeId() << '\n';
        items_[i]->save(out);
    }
    out.flush();
    if (!out) throw std::runtime_error("Failed to write data to file.");
}

void Keeper::loadFromFile(const std::string& filename) {
    std::ifstream in(filename.c_str());
    if (!in) throw std::runtime_error("File not found or cannot be opened.");

    std::string header;
    std::getline(in, header);
    if (header != "BOOKSTORE_LAB1_V1") {
        throw std::runtime_error("This is not a valid bookstore data file.");
    }
    int amount;
    if (!(in >> amount) || amount < 0) {
        throw std::runtime_error("Invalid number of products in file.");
    }

    Keeper loaded;  // Keep old products untouched if reading fails.
    for (int i = 0; i < amount; ++i) {
        int type;
        if (!(in >> type)) throw std::runtime_error("Missing product type.");
        Base* item = nullptr;
        if (type == 1) item = new Book();
        else if (type == 2) item = new Textbook();
        else if (type == 3) item = new Stationery();
        else throw std::runtime_error("Unknown product type in file.");

        try {
            item->load(in);
            loaded.add(item);
        } catch (...) {
            delete item;
            throw;
        }
    }
    swapWith(loaded);
```

```
}
```

Файл Input.h

```
#ifndef INPUT_H
#define INPUT_H

#include <string>

std::string readText(const std::string& prompt);
int readInt(const std::string& prompt, int minimum, int maximum);
double readPrice(const std::string& prompt);

#endif
```

Файл Input.cpp

```
#include "Input.h"

#include <algorithm>
#include <cmath>
#include <iostream>
#include <sstream>
#include <stdexcept>

std::string readText(const std::string& prompt) {
    std::string value;
    while (true) {
        std::cout << prompt;
        if (!std::getline(std::cin, value)) {
            throw std::runtime_error("Input was interrupted.");
        }
        if (value.find_first_not_of(" \t") != std::string::npos) {
            return value;
        }
        std::cout << "The field cannot be empty. Try again.\n";
    }
}

int readInt(const std::string& prompt, int minimum, int maximum) {
    while (true) {
        std::string line;
        std::cout << prompt;
        if (!std::getline(std::cin, line)) {
            throw std::runtime_error("Input was interrupted.");
        }
        std::istringstream input(line);
        int number;
        char extra;
        if ((input >> number) && !(input >> extra) &&
            number >= minimum && number <= maximum) {
            return number;
        }
        std::cout << "Enter an integer from " << minimum
                  << " to " << maximum << ".\n";
    }
}

double readPrice(const std::string& prompt) {
    while (true) {
        std::string line;
        std::cout << prompt;
        if (!std::getline(std::cin, line)) {
            throw std::runtime_error("Input was interrupted.");
        }
        std::replace(line.begin(), line.end(), ',', '.');
        std::istringstream input(line);
        double number;
        char extra;
        if ((input >> number) && !(input >> extra) &&
            std::isfinite(number) && number >= 0.0) {
            return number;
        }
        std::cout << "Enter a non-negative price (example: 129.50).\n";
    }
}
```

Файл main.cpp

```
#include "Base.h"
#include "Book.h"
#include "Input.h"
#include "Keeper.h"
#include "Stationery.h"
#include "Textbook.h"

#include <climits>
#include <exception>
#include <iostream>

#ifdef _WIN32
#include <windows.h>
#endif

void printMenu() {
    std::cout << "\n========== BOOKSTORE (VARIANT 8) ==========\n"
              << "1. Add book\n"
              << "2. Add textbook\n"
              << "3. Add stationery\n"
              << "4. Show all products\n"
              << "5. Show one product\n"
              << "6. Edit product field\n"
              << "7. Delete product\n"
              << "8. Save all to file\n"
              << "9. Load all from file\n"
              << "10. Demonstrate copying\n"
              << "0. Exit\n";
}

int getProductIndex(const Keeper& keeper) {
    if (keeper.size() == 0) {
        std::cout << "The shop is empty.\n";
        return -1;
    }
    return readInt("Product number: ", 1, keeper.size()) - 1;
}

void addProduct(Keeper& keeper, int type) {
    Base* item = nullptr;
    if (type == 1) item = new Book();
    if (type == 2) item = new Textbook();
    if (type == 3) item = new Stationery();
    try {
        item->input();
        keeper += item;
        item = nullptr; // Keeper now owns this pointer.
        std::cout << "Product added.\n";
    } catch (...) {
        delete item;
        throw;
    }
}

void demonstrateCopy(const Keeper& keeper) {
    std::cout << "Making independent copy of Keeper...\n";
    Keeper copied(keeper);  // Copy constructor, deep copy via clone().
    std::cout << "Copied objects: " << copied.size() << "\n";
    std::cout << "Making another Keeper and assigning the copy...\n";
    Keeper second(1);       // Constructor with parameters.
    second = copied;        // Overloaded assignment operator.
    std::cout << "Assigned objects: " << second.size() << "\n";
    std::cout << "Copies are now destroyed at end of function.\n";
}

int main() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif
    Keeper keeper;
    try {
        while (true) {
```

```
            printMenu();
            int choice = readInt("Your choice: ", 0, 10);
            try {
                if (choice == 0) break;
                if (choice >= 1 && choice <= 3) {
                    addProduct(keeper, choice);
                } else if (choice == 4) {
                    keeper.showAll();
                } else if (choice == 5) {
                    int index = getProductIndex(keeper);
                    if (index >= 0) std::cout << keeper[index];
                } else if (choice == 6) {
                    int index = getProductIndex(keeper);
                    if (index >= 0) {
                        keeper[index].edit();
                        std::cout << "Field updated.\n";
                    }
                } else if (choice == 7) {
                    int index = getProductIndex(keeper);
                    if (index >= 0) {
                        keeper.remove(index);
                        std::cout << "Product deleted.\n";
                    }
                } else if (choice == 8) {
                    keeper.saveToFile(readText("File path for saving: "));
                    std::cout << "All products saved.\n";
                } else if (choice == 9) {
                    keeper.loadFromFile(readText("File path for loading: "));
                    std::cout << "All products loaded.\n";
                } else if (choice == 10) {
                    demonstrateCopy(keeper);
                }
            } catch (const std::exception& error) {
                std::cout << "Error: " << error.what() << '\n';
            }
        }
    } catch (const std::exception& error) {
        std::cerr << "Input ended or critical error: " << error.what() << '\n';
        return 1;
    }
    std::cout << "Goodbye!\n";
    return 0;
}
```